request

DrawTrainCardRequest

ClaimRouteRequest

KeepDrawDestRequest

DrawDestCardRequest

<<interface>>
ActionRequest

<<interface>>
GameUI

save

GameMemento

GameStateMapper

+ createMemento(game: Game): GameMemento
+ restore(gameMemento: GameMemento): Game

contains 1 1..*

PlayerMemento

RouteMemento

uses 1 1

GameSaveService

- serializer: GameSerializer
- stateMapper: GameStateMapper

+ save(game: Game, filePath: String): void
+ load(filePath: String): Game

has 1 1

GameSerializer

+ serialize(game: GameMemento): String
+ deserialize(data: String): GameMemento

setup

PlayerConfiguration

- playerAbility: PlayerAbility

+ getPlayerAbility: PlayerAbility

1 includes 2 **GameConfiguration**

<<enumeration>>
MapType

LONDON

1 selects 1

GameSetupDirector

LondonGameBuilder

<<interface>>
GameBuilder

main

has 1 1

<<interface>>
PendingDecisionVisitor

GameController

- game: Game
- gameSaveService: GameSaveService

+ saveGame(filePath: String): void
+ loadGame(filePath: String): void

1 manages 1

Game

contains 1 1

command

contains 1 1

<<enumeration>>
ActionCost

FREE
FULL
HALF
- cost: int
+ getCost(): int

<<interface>>
ActionCommand

ActionResult

1 has 0..1

<<interface>>
PendingDecision

0..1 0..1

1 has 0..1

<<enumeration>>
EndTurnOutcome

NEXT_PLAYER_TURN
LAST_ROUND_TRIGGERED
LAST_ROUND_CONTINUES
GAME_OVER

1 with status 0..1

<<enumeration>>
ActionStatus

SUCCESS
FAILED
PENDING
PENDING_FAILED

TicketDecision

ClaimRouteCommand

DrawTrainCardCommand

InvalidCommand

KeepDestCommand

DrawDestCommand

processes

draws

updates

updates

updates

records

refers

finalizes scores

endgame

<<interface>>
RouteScoringStrategy

EndGameService

contains contains

StandardRouteScoring

<<interface>>
EndGameScoringStrategy

<<interface>>
TieBreakerStrategy

DestinationTicketScoring

LongestRouteScoring

character

Player

- ability: PlayerAbility
- doubleTurn: boolean

+ claimRoute(Route, List<TrainCard>, RouteScoringStrategy): void
+ getAbility(): PlayerAbility
+ activateDoubleTurn(): void
+ consumeDoubleTurn(): void
+ isDoubleTurn(): boolean

has 1 1

has 1 0..1

<<enumeration>>
PlayerColour

RED
BLUE
GREEN
YELLOW
BLACK

PlayerHand

holds

holds

card

uses

claims

<<abstract>>
Card

0..* holds

TrainCard

DestinationCard

0..* 0..*

0..* 0..*

has 1 1

<<enumeration>>
CardColour

ORANGE
YELLOW
GREEN
BLUE
PINK
BLACK
WILD

holds

deck

<<abstract>>
Deck

1 1 1 1

TrainCardDeck

- discardedPile: List<TrainCard>
- faceUpPile: List<TrainCard>

- checkAndReplaceLocomotives(): void
+ conditionHasTooManyLocomotive(): boolean
+ conditionCanReplaceFaceUp(): boolean
+ discardFacePile(): void
+ refillFaceUp(): void

DestinationCardDeck

holds

tracks

ability

has

has

<<interface>>
PlayerAbility

+ apply(): void
+ getName(): string

FerryMasterAbility

- FREE_CLAIM: int = 2 {readOnly}
- freeClaimRemaining: int

ColourMasterAbility

- BONUS: int = 2 {readOnly}
- UNIQUE_THRESHOLD: int = 4 {readOnly}
- claimedColour: Set<RouteColour>

has

claimed by

map

references

Board

1 contains 1..*

City

contains

connects

Route

- ferry: boolean

+ isFerry(): boolean
+ isOwnedBy(player: Player): boolean
+ getOtherCity(cityA: City): City

claimed with

has

<<enumeration>>
RouteColour

ORANGE
YELLOW
GREEN
BLUE
PINK
BLACK
GREY

contains

RouteGroup

strategy

<<interface>>
RouteClaimStrategy

+ validate(player: Player, route: Route, spentCards: List<TrainCard>): boolean

FerryRouteClaimStrategy

StandardRouteClaimStrategy

GrayRouteClaimStrategy

turnmanager

TurnMediator

+ allowBonusTurn(): void

LastRoundTracker

triggered by